

Introduction to Computer Vision using OpenCV

OpenCV is one of the most important libraries used in robotics. Due to its image processing utilities, it allows our bots to interact with the real world and perform functions like detection, recognition, image analysis, video processing and ML implementation.

1. OpenCV Installation Guide (Python)

Python 3 Installation

Install Python 3 on your system. You can download the installer from the official Python website or use a package manager.

Windows/macOS:

Download the installer from the official Python website. Make sure to check the box to "Add Python to PATH" during installation.

Linux (Debian/Ubuntu):

Shell

```
sudo apt update  
sudo apt install python3
```

PIP Installation

PIP is the package installer for Python. It is usually included with Python 3, but if it's missing, you can install it using:

Shell

```
sudo apt install python3-pip
```

Prerequisites

- Python
- pip
- numpy

Installation Steps

To install the required packages, run the following commands in your terminal:

```
pip install numpy==1.21.5
pip install opencv-contrib-python==4.7.0.72
```

Verification

Run the following Python script to verify the installation:

Python

```
import cv2
import numpy as np
print(np.__version__)
print(cv2.__version__)
```

A successful installation will print the version number of OpenCV to the console.

2. Numpy and OpenCV co-relation

NumPy and OpenCV are closely related because OpenCV stores and processes images as NumPy arrays. NumPy provides fast numerical operations, while OpenCV uses these arrays for image and video processing. Every digital image can be represented as a matrix of pixel values. In OpenCV, an image is treated as a NumPy array.

- A grayscale image is represented as a 2D matrix
- A color image is represented as a 3D matrix

For a color image:

- First dimension → Height (rows)
- Second dimension → Width (columns)
- Third dimension → Color channels (B, G, R)

Thus, every image can be viewed as a 3D matrix where each matrix element contains an array of 3 pixel values representing: [B, G, R]

Where:

- B → Blue intensity
- G → Green intensity
- R → Red intensity

Each pixel value ranges from: 0 to 255

- 0 → No intensity
- 255 → Maximum intensity

2.1. Accessing and Modifying Pixel Values

NumPy's array indexing allows you to access and modify individual pixel values.

Accessing Pixels

Python

```
import cv2
import numpy as np

img = cv2.imread("image.jpg")

pixel = img[100, 200]
print(pixel)
# Output: [B, G, R]
```

Modifying Pixel Values

```
img[100, 200] = [0, 255, 0]
```

This changes the selected pixel to green.

3. Basic functions of OpenCV

1. Reading and Displaying an Image

Use `cv2.imread()` to load an image from a file and `cv2.imshow()` to display it in a window.

Python

```
img = cv2.imread("image.jpg")
cv2.imshow("Display Window", img)
cv2.waitKey(0)
cv2.destroyAllWindows()
```

2. Saving an Image

To save an image to your local system, use `cv2.imwrite()`.

Python

```
cv2.imwrite("output_image.png", img)
```

3. Resizing an Image

Scale an image to new dimensions using `cv2.resize()`.

Python

```
resized_img = cv2.resize(img, (width, height))
```

4. Cropping an Image

Since images are represented as NumPy arrays, you can crop them using standard array slicing.

Python

```
cropped_img = img[y_start:y_end, x_start:x_end]
```

5. Converting Color Space

Convert an image from BGR to HSV color space using `cv2.cvtColor()`.

Python

```
hsv_img = cv2.cvtColor(img, cv2.COLOR_BGR2HSV)
```

6. Drawing Shapes

OpenCV provides functions to draw various shapes directly onto an image.

6.1. Drawing a Rectangle

Use `cv2.rectangle()` to draw a rectangular shape on an image.

Python

```
cv2.rectangle(img, (50,50), (200,200), (0,255,0), 2)
```

6.2. Drawing a Circle

Use `cv2.circle()` to draw a circular shape on an image.

Python

```
cv2.circle(img, (150,150), 50, (255,0,0), 3)
```

6.3. Drawing a Line

Use `cv2.line()` to draw a straight line on an image.

Python

```
cv2.line(img, (0,0), (300,300), (0,0,255), 2)
```

4. Object detection and Tracking

Object Detection and Tracking using OpenCV provides powerful techniques for detecting and tracking moving or static objects in images and videos. Object detection identifies the object,

while object tracking follows the movement of the detected object across video frames. Object detection and tracking are widely used in:

- Surveillance systems
- Autonomous vehicles
- Robotics
- Traffic monitoring
- Face tracking
- Sports analytics

Object Detection in OpenCV

Object detection refers to identifying the location of an object in an image or video.

Common Detection Techniques

- Haar Cascade Detection
- Contour Detection
- Color-Based Detection
- Deep Learning Detection
- Background Subtraction

1. Haar Cascade Object Detection

Haar Cascade classifiers are pre-trained models used for detecting objects such as faces, eyes, and cars.

Necessary Functions

Loading Cascade Classifier

Python

```
cascade = cv2.CascadeClassifier(
    "haarcascade_frontalface_default.xml")
```

Function

- `cv2.CascadeClassifier()` Loads trained classifier

Detecting Objects

Python

```
objects = cascade.detectMultiScale(
    gray,
    scaleFactor=1.1,
    minNeighbors=5)
```

Function

- `detectMultiScale()` Detects objects in image

Parameters

- scaleFactor Reduces image size at each scale
- minNeighbors Detection accuracy control

Drawing Bounding Box

Python

```
for (x, y, w, h) in objects:
    cv2.rectangle(img,
    (x, y),
    (x+w, y+h),
    (0,255,0),
    2)
```

Function

- cv2.rectangle() Draws detection box

Complete Face Detection Program

Python

```
import cv2

face_cascade = cv2.CascadeClassifier(
    "haarcascade_frontalface_default.xml")

img = cv2.imread("person.jpg")
gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)

faces = face_cascade.detectMultiScale(
    gray,
    1.1,
    5)

for (x, y, w, h) in faces:
    cv2.rectangle(img,
    (x, y),
    (x+w, y+h),
    (255,0,0),
    2)

cv2.imshow("Face Detection", img)
cv2.waitKey(0)
cv2.destroyAllWindows()
```

Object Tracking in OpenCV

Object tracking continuously follows an object in video frames after detection.

Types of Object Tracking Algorithms

OpenCV provides several tracking algorithms:

Tracker	Description
BOOSTING	Basic tracking
MIL	Better accuracy
KCF	Fast tracking
CSRT	High accuracy
MOSSE	Very fast
MedianFlow	Predictable motion

2. Creating an Object Tracker

CSRT Tracker Example

Python

```
tracker = cv2.TrackerCSRT_create()
```

Function

- `cv2.TrackerCSRT_create()` Creates CSRT tracker

3. Capturing Video

Python

```
cap = cv2.VideoCapture(0)
```

Function

- `cv2.VideoCapture()` Opens webcam/video

4. Selecting Object for Tracking

Python

```
ret, frame = cap.read()
```

```
bbox = cv2.selectROI(  
    "Tracking",  
    frame,  
    False)
```

```
tracker.init(frame, bbox)
```

Functions

- cv2.selectROI() Selects object manually
- tracker.init() Initializes tracking

5. Updating Tracker

Python

```
success, bbox = tracker.update(frame)
```

Function

- tracker.update() Tracks object in next frame

6. Drawing Tracking Rectangle

Python

if success:

```
x, y, w, h = [int(v) for v in bbox]
```

```
cv2.rectangle(frame,  
(x, y),  
(x+w, y+h),  
(0,255,0),  
2)
```

Complete Object Tracking Program

Python

```
import cv2
```

```
cap = cv2.VideoCapture(0)
```

```
tracker = cv2.TrackerCSRT_create()
```

```
ret, frame = cap.read()
```

```
bbox = cv2.selectROI(  
"Tracking",  
frame,  
False)
```



```

tracker.init(frame, bbox)

while True:

    ret, frame = cap.read()

    success, bbox = tracker.update(frame)

    if success:
        x, y, w, h = [int(v) for v in bbox]

        cv2.rectangle(frame,
            (x, y),
            (x+w, y+h),
            (0,255,0),
            2)

        cv2.imshow("Object Tracking", frame)

        key = cv2.waitKey(1)

        if key == 27:
            break

    cap.release()
    cv2.destroyAllWindows()

```

Color-Based Object Detection and Tracking

Objects can also be detected using color ranges in HSV color space.

Converting to HSV

```

Python
hsv = cv2.cvtColor(frame, cv2.COLOR_BGR2HSV)

```

Function

- `cv2.cvtColor()` Converts image format

Defining Color Range

Python

```
lower = np.array([35, 50, 50])  
upper = np.array([85, 255, 255])
```

Creating Mask

Python

```
mask = cv2.inRange(hsv, lower, upper)
```

Function

- `cv2.inRange()` Extracts selected color

Finding Contours

Python

```
contours, _ = cv2.findContours(  
mask,  
cv2.RETR_TREE,  
cv2.CHAIN_APPROX_SIMPLE)
```

Function

- `cv2.findContours()` Detects object boundaries

Resources:-

▶ [OpenCV Course - Full Tutorial with Python](#)

▶ [OpenCV Python Course — Learn Computer Vision and AI](#)

For object tracking:-

▶ [OpenCV Python Tutorial For Beginners 13 - Object Detection and Object Tracking Using ...](#)

▶ [How to Detect Colors in OpenCV \[Python\]](#)